Image Classification with CNNs

A summary interface to explore the models

Prepared by: Wijdane AARROUB
Supervised by: Pr. Asmae OUHMIDA
Program: Master Distributed Systems (SDIA), ENSET Mohammedia
Academic year: 2026/2027 · November 8, 2026

1. Purpose of the interface

The training code and the models were already completed. This document adds the last deliverable: a single Streamlit interface that summarizes the work and lets the reader reproduce the main steps of the project without opening a notebook:

- choose the initial CNN and look at its results;
- switch to the improved CNN and look at its results;
- display the curves and everything that describes the model (architecture, parameters, optimizer);
- compare the models, and classify the same image with both CNNs.

Every figure below is a real screenshot of the application running locally (localhost:8501). All numbers displayed by the interface come from the saved .keras models, evaluated on the 3,000 images of the Intel test split.

2. Structure of the interface

The model is chosen once in the sidebar; six tabs then describe it. The same layout is used for every model, so switching from the initial to the improved CNN immediately shows what changed.

Tab	What it shows
Overview	Selected model, headline metrics (test accuracy, macro F1, parameters, classes), usage guide and the list of project models.
Predict	Classify a test-set example or one or several uploaded images: predicted class, confidence, true class, top-k probabilities, low-confidence warning, preview of the resized image.
Test results	Accuracy, macro F1, test loss, confusion matrix, F1 per class, most frequent confusions, precision/recall table.
Curves	ROC curve per class with AUC, calibration curve, confidence distribution, and the training history (loss/accuracy per epoch) when available.
Model details	Total/trainable parameters, input size, optimizer and learning rate, loss, epochs, training time, layer table and parameters per layer.
Compare models	Table and charts for all models, F1 per class for initial vs improved, and both CNNs on the same image.

This interface is the fusion of the generic classification application of the previous report and the summary tabs. From the generic application it keeps: automatic configuration (input size read from the model, class names read from the <model>_classes.json file or typed in the sidebar, pixel scaling auto-detected), upload of any .keras model, upload of several images at once, the top-k and low-confidence-threshold settings, the "what the model sees" preview and robust preprocessing (EXIF rotation, transparent PNGs). The summary tabs are added on top. The application is started with streamlit run streamlit_cnn.py (dependencies: streamlit, tensorflow, pillow, numpy, matplotlib, pandas).

3. Walkthrough

3.1 Overview

The opening page presents the selected model with its key figures and explains the six steps of the demonstration. The initial CNN is selected first: 88.10 % test accuracy, 88.24 % macro F1 and 101,894 parameters.

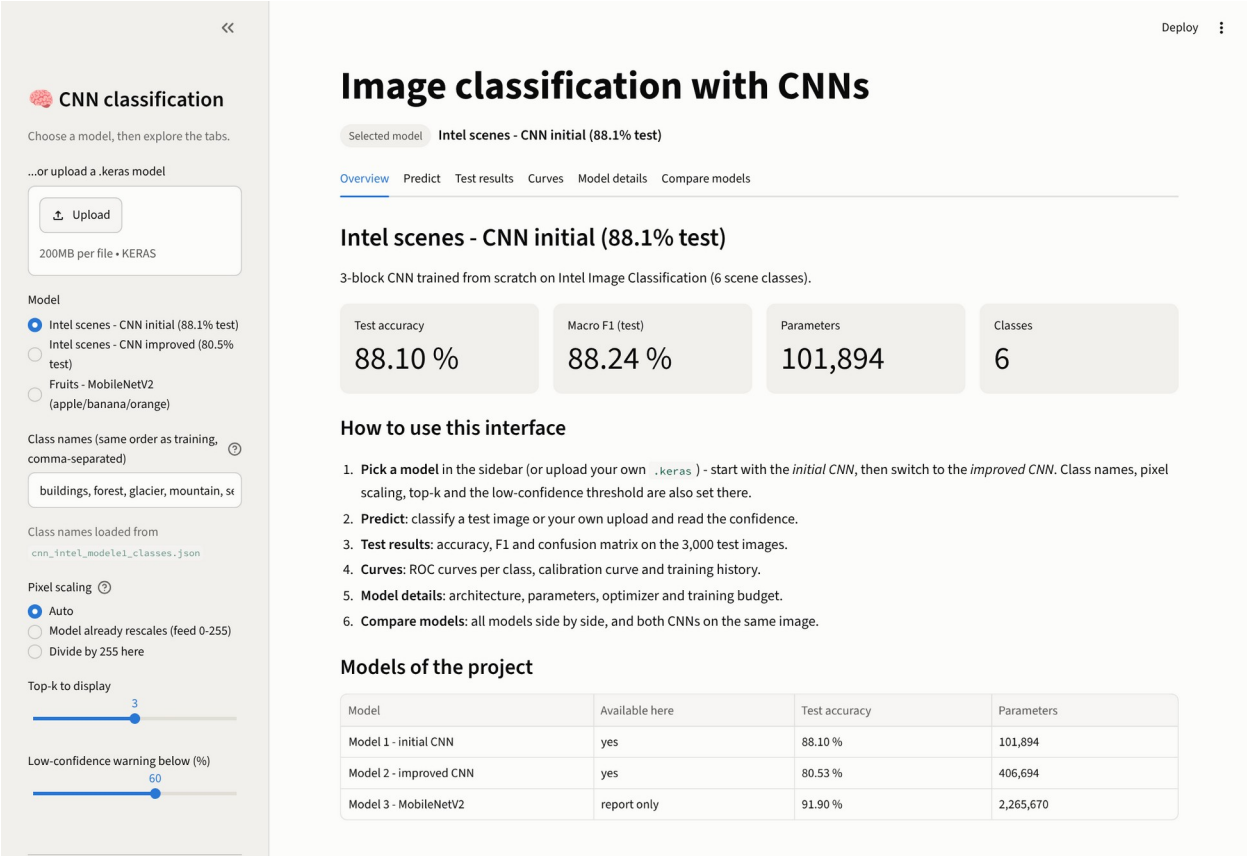


Figure 1 - Overview tab with the initial CNN selected.

3.2 Initial CNN

Prediction

A mountain image taken from the test set is classified as mountain with 93.7 % confidence; the remaining probability goes to glacier (6.3 %), the class most often confused with mountain. The interface also shows the true class and whether the prediction is correct.

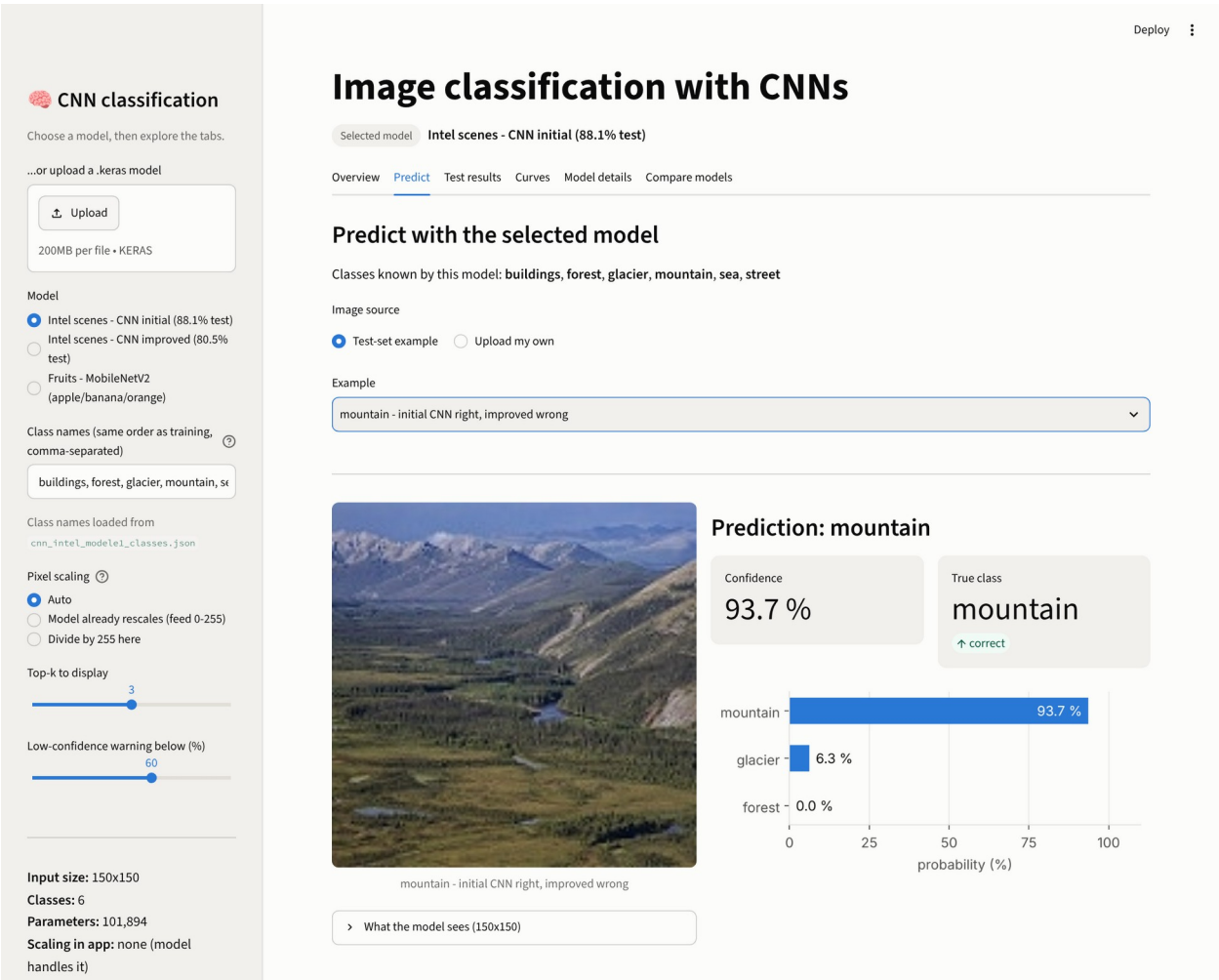


Figure 2 - Predict tab, initial CNN: correct prediction (mountain, 93.7 %).

Test results

On the 3,000 test images the initial CNN reaches 88.10 % accuracy and 88.24 % macro F1 (test loss 0.353), exactly the values of the report. It makes 357 errors. The confusion matrix shows where they come from: mountain/glacier (76 + 50 images) and buildings/street (48 + 42 images) together account for 216 of the 357 errors (about 60 %). Forest is the easiest class (F1 98.1 %), mountain the hardest (82.4 %).

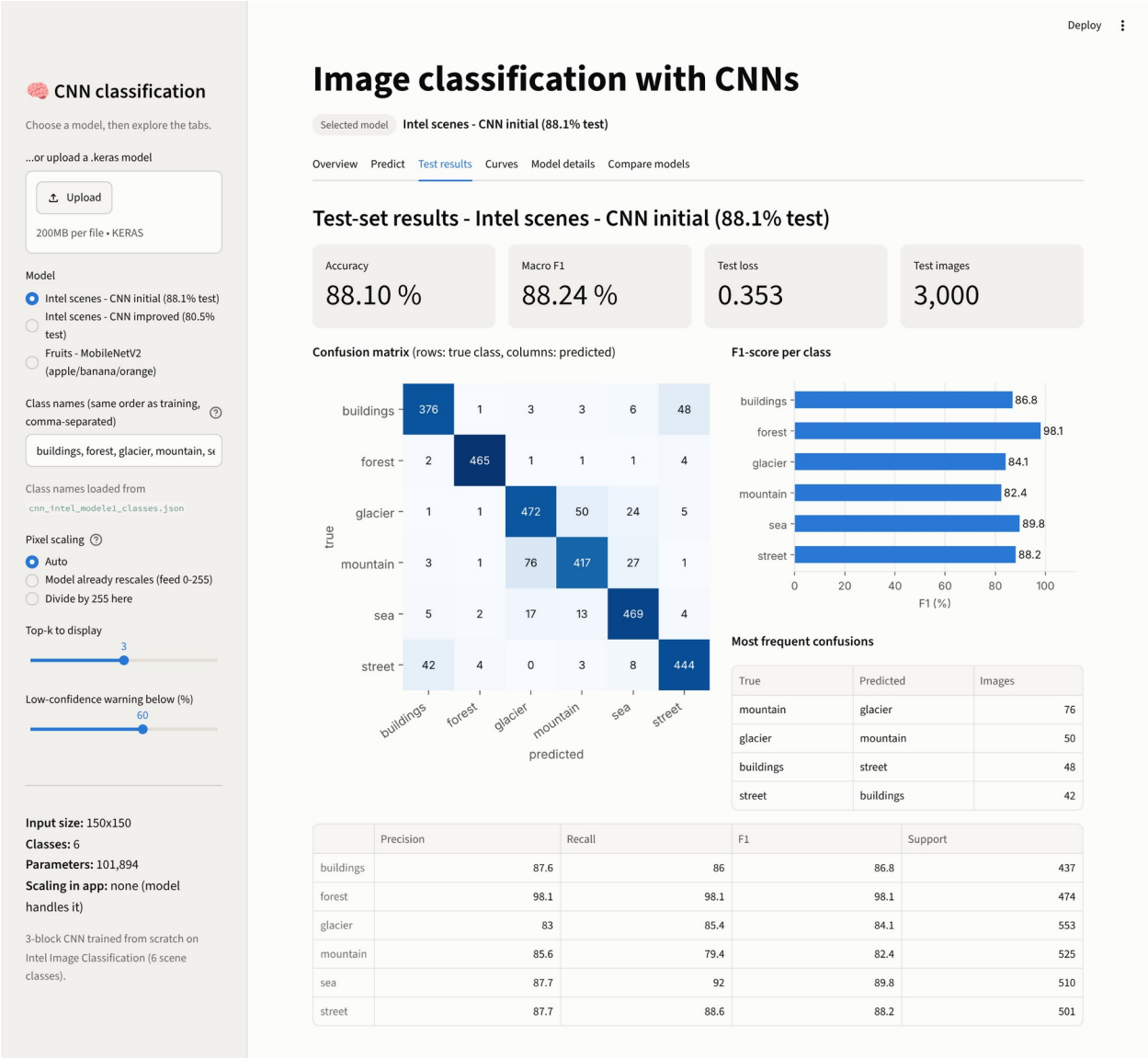


Figure 3 - Test results tab, initial CNN: metrics, confusion matrix, F1 per class and main confusions.

Curves

The ROC curves are close to the top-left corner for every class (macro AUC 0.985); forest is almost perfect (AUC 0.999) and glacier is the lowest (0.973). The calibration curve and the confidence histogram show how far the confidence can be trusted: 68.1 % of the test images are predicted with at least 90 % confidence, and 96.4 % of those are correct. The same tab hosts the training history (loss and accuracy per epoch); see the note in section 5.

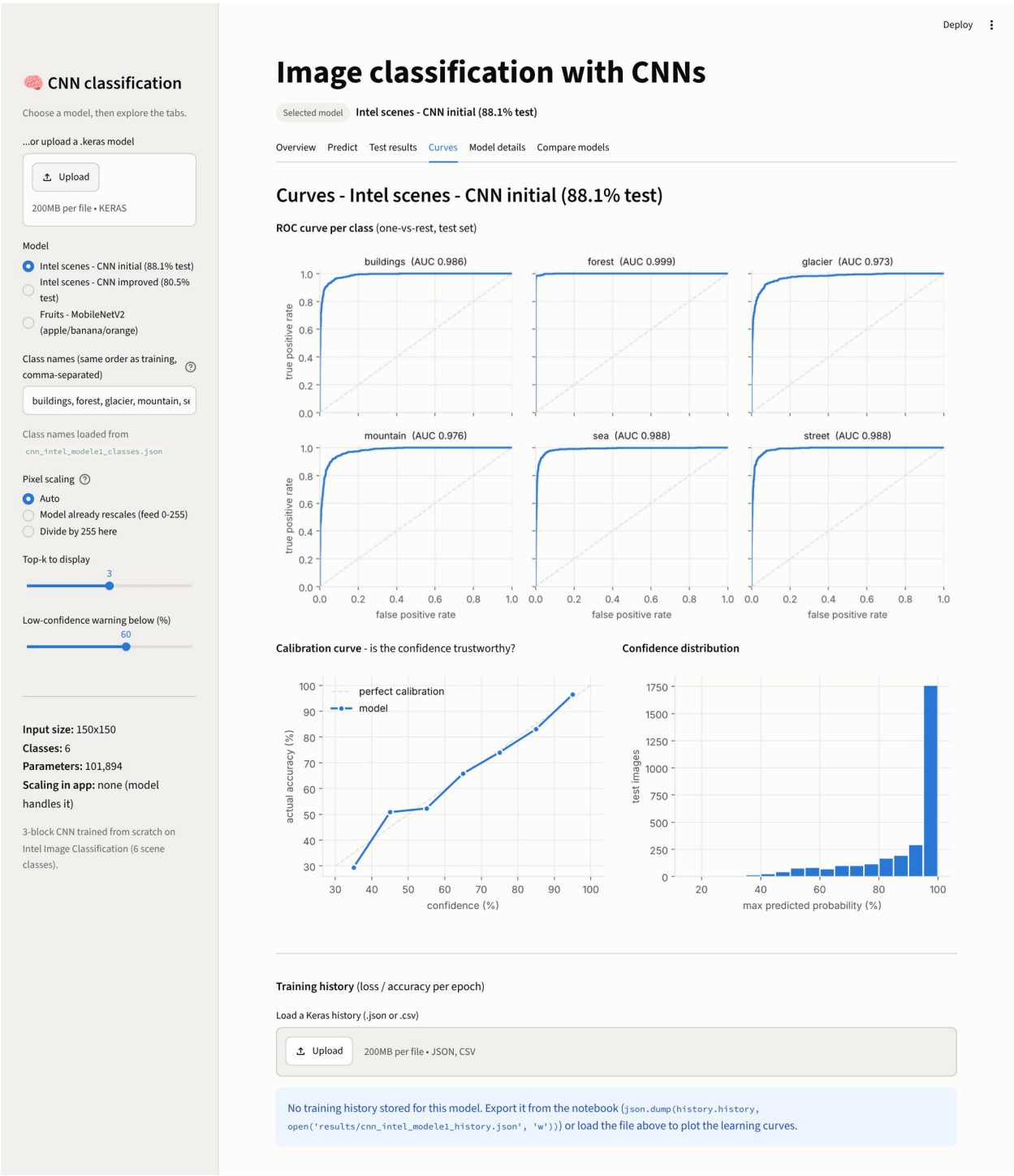


Figure 4 - Curves tab, initial CNN: ROC per class, calibration curve, confidence distribution.

Model details

This tab lists the architecture layer by layer (3 convolutional blocks with max-pooling, global average pooling and a dense head, with a Rescaling layer inside the model), the 101,894 parameters, the Adam optimizer with a learning rate of 1.25e-04, and the training budget of the report (40 epochs, 7.2 minutes, 88.06 % validation accuracy).

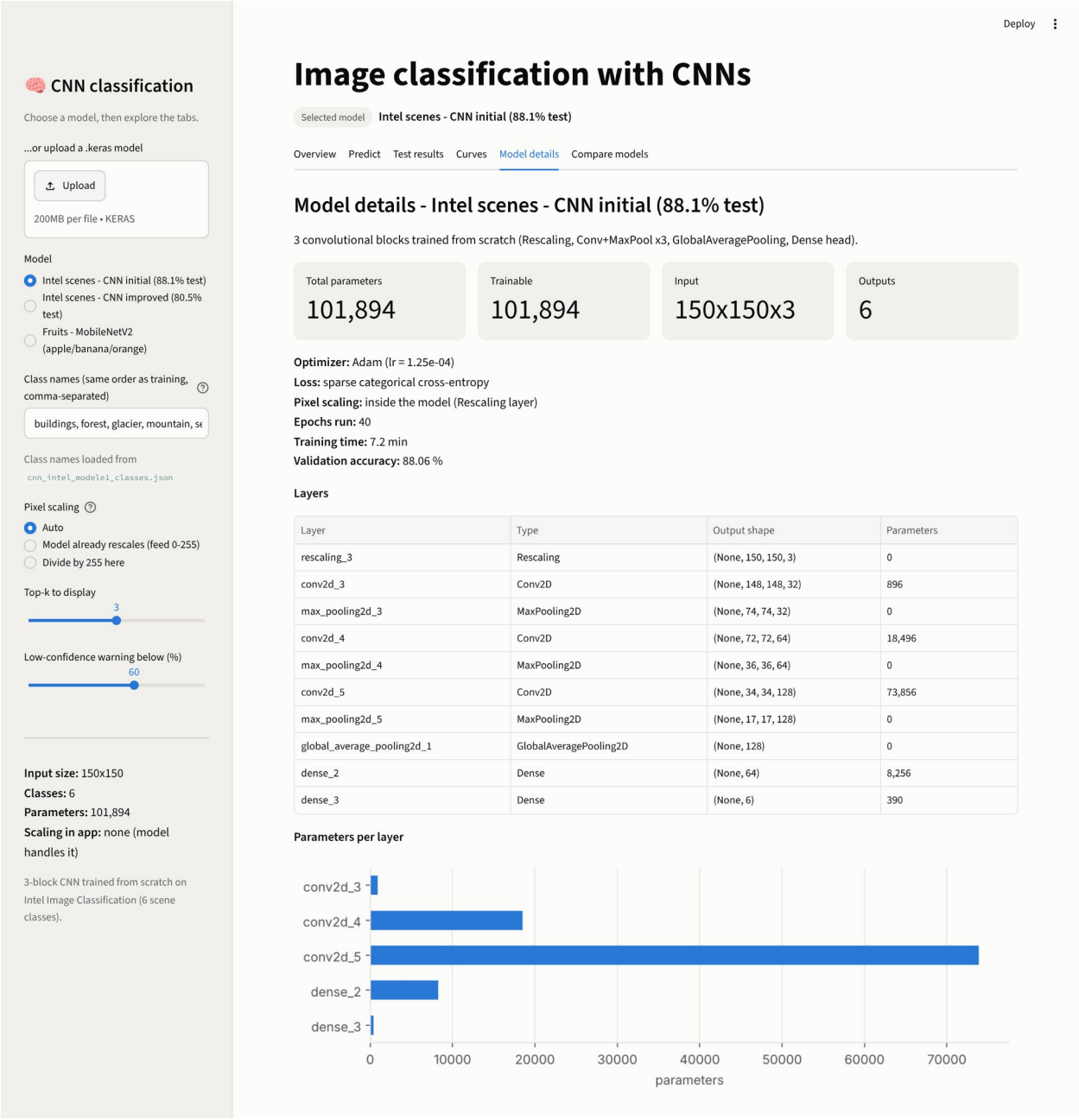


Figure 5 - Model details tab, initial CNN.

3.3 Improved CNN

Selecting the improved CNN in the sidebar re-computes every tab. This model adds a fourth convolutional block, batch normalization, dropout and data augmentation (406,694 parameters, Adam with a learning rate of 2.5e-04).

Prediction

On the same mountain image the improved CNN answers sea with 66.2 %, while the correct class (mountain) only gets 29.4 %. The interface flags the answer as wrong. This is a typical example of the weaker behaviour of Model 2.

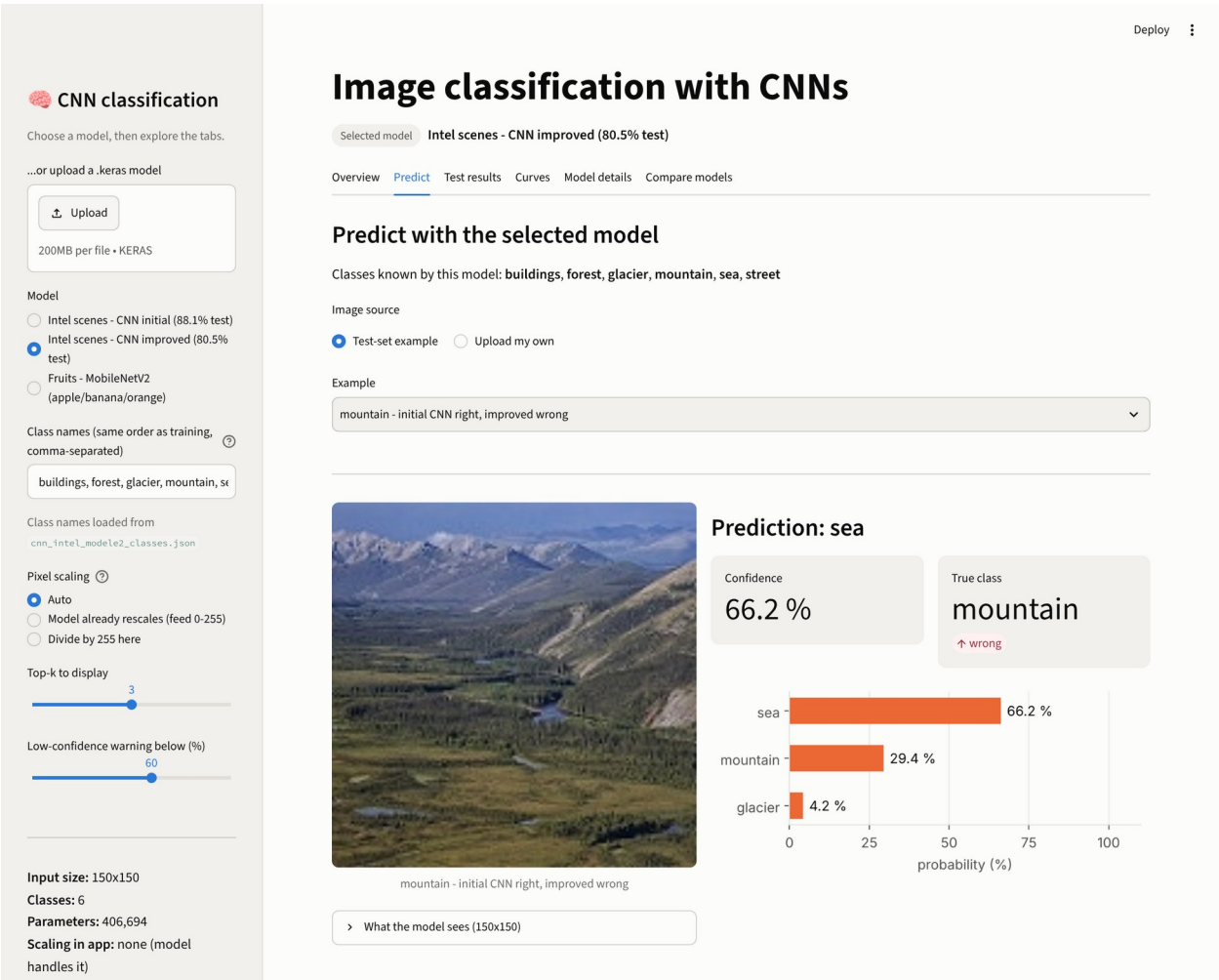


Figure 6 - Predict tab, improved CNN: the same image is misclassified as sea (66.2 %).

Test results

The improved CNN reaches 80.53 % accuracy and 80.63 % macro F1 (test loss 0.538), i.e. 7.6 points below the initial CNN, with 584 errors instead of 357. Its main confusions are different: mountain and glacier images are often predicted as sea (109 and 108 images), which lowers the precision of the sea class (F1 76.5 %). Glacier (73.9 %) and mountain (74.0 %) are the weakest classes.

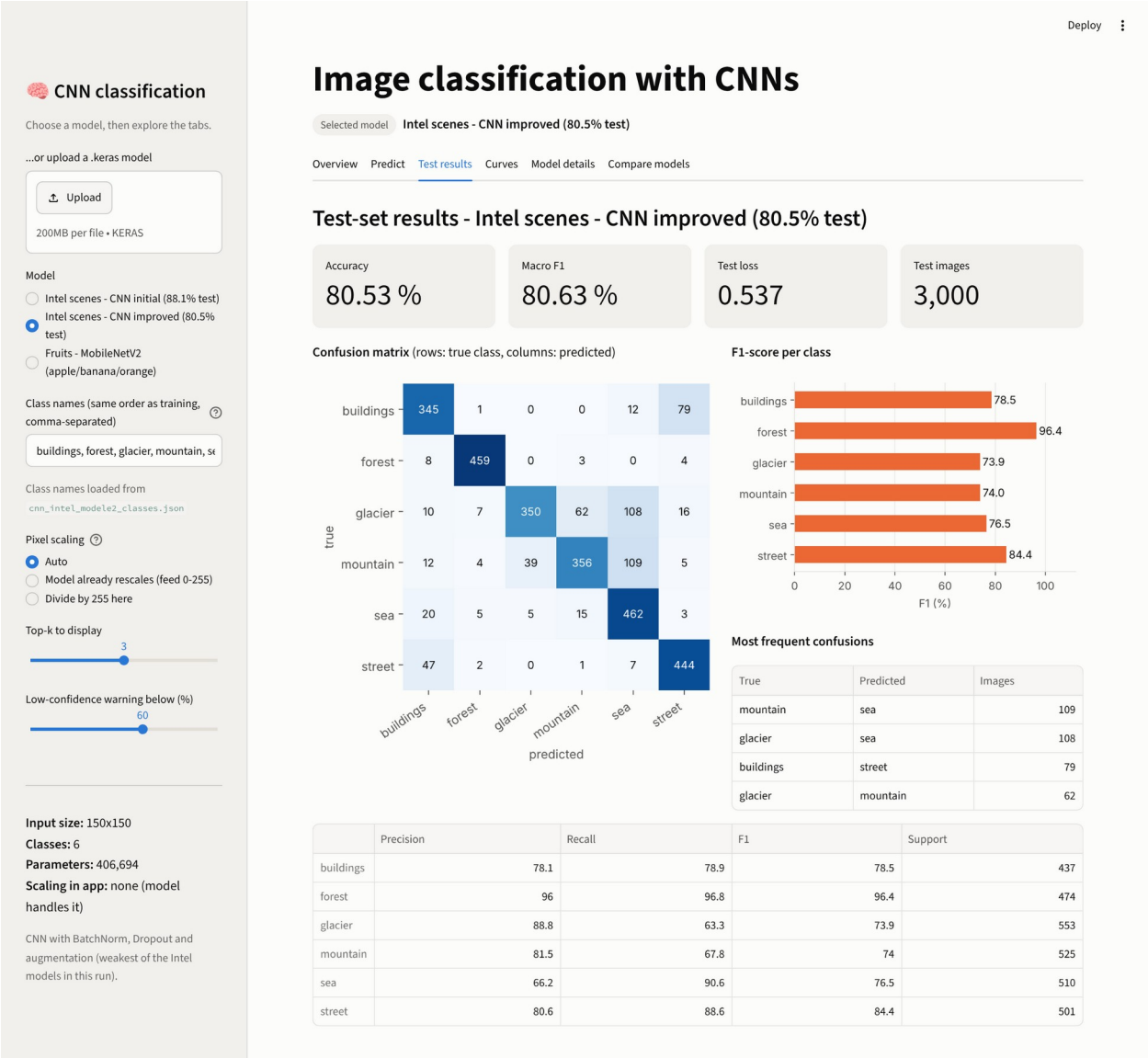


Figure 7 - Test results tab, improved CNN.

Curves

The ROC curves remain good (macro AUC 0.972 versus 0.985), so the classes are still well ranked, but less sharply than with the initial CNN. The model is also less decisive: only 55.7 % of the test images reach 90 % confidence (initial CNN: 68.1 %), with 93.8 % accuracy among them. In the calibration curve the points sit slightly below the diagonal at high confidence, a mild over-confidence.

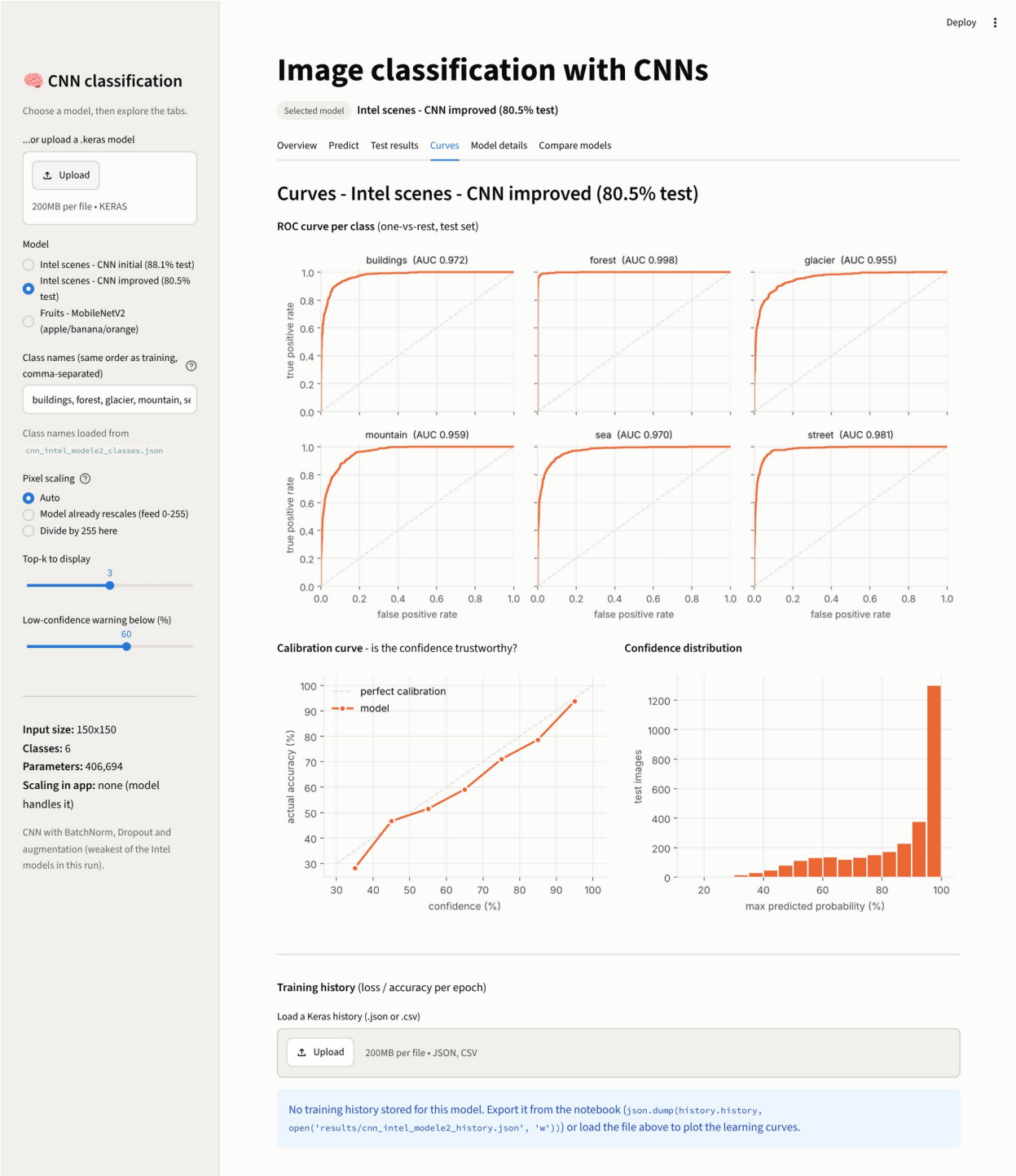


Figure 8 - Curves tab, improved CNN.

Model details

The layer table shows the added blocks (Conv2D, BatchNormalization, Activation, MaxPooling, Dropout) and the data augmentation block at the input. The parameter chart shows that most of the 406,694 parameters sit in the last convolution (conv2d_9). Training stopped after 19 epochs (early stopping), in 9.8 minutes, with 81.75 % validation accuracy.

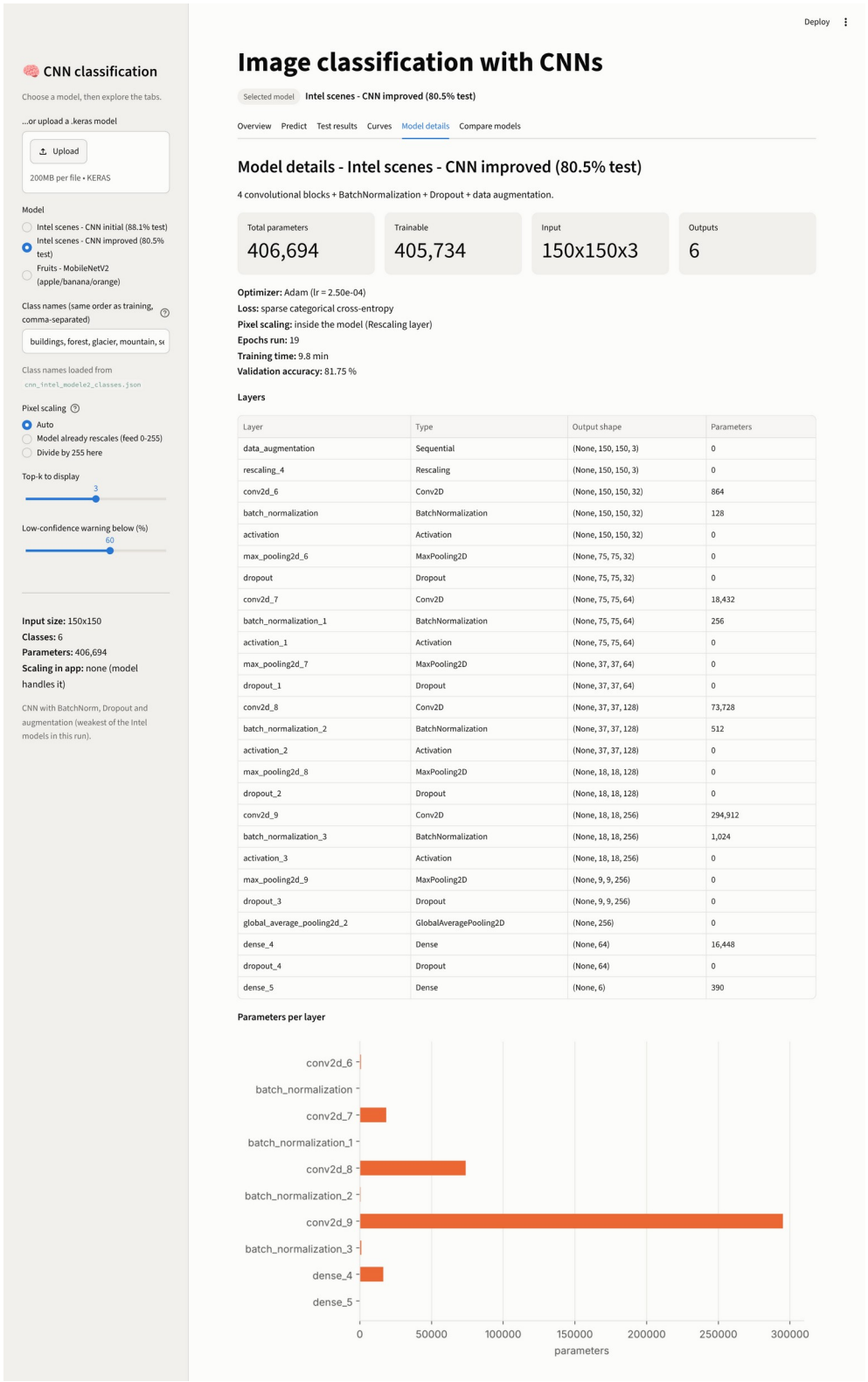


Figure 9 - Model details tab, improved CNN.

3.4 Comparing the models

The comparison tab puts all the Intel models side by side. The table and the bar chart reproduce the figures of the report; Model 3 (MobileNetV2, 91.90 % test accuracy) is shown for reference, but its .keras file is not in the project folder, so it cannot be selected or evaluated here. The per-class chart shows that the initial CNN is better than the improved CNN on every class. At the bottom, the same image is classified by both CNNs at once, with the top-3 probabilities of each.

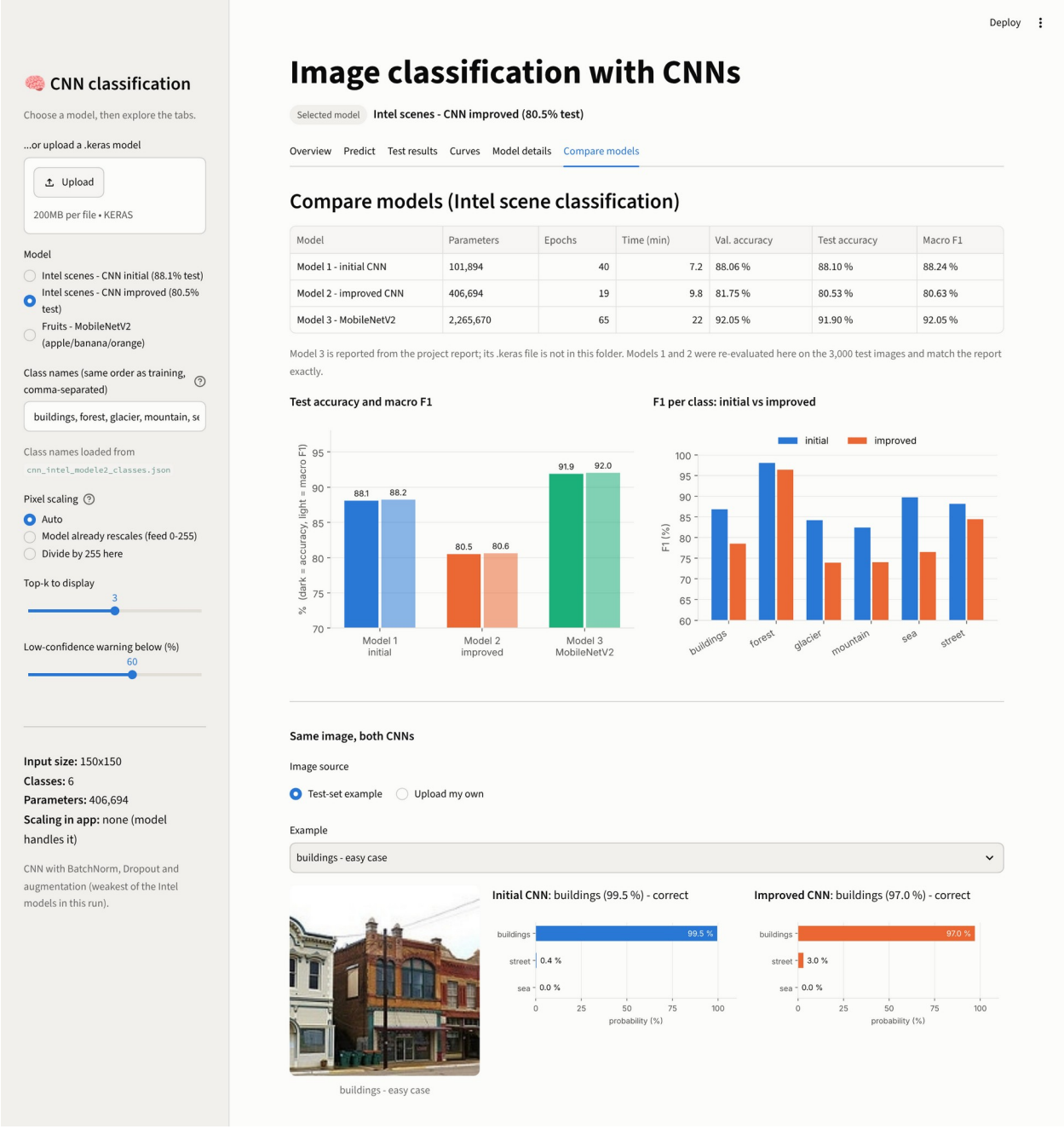


Figure 10 - Compare models tab: summary table, accuracy/F1 charts, F1 per class and same-image comparison.

3.5 Fruit model

The same interface works for the fruit classifier (MobileNetV2, 128x128 input, classes apple, banana, orange) without any code change: the input size and class names are read from the model and its JSON file. An apple image is classified as apple with 98.2 % confidence. This is an illustrative example, not an evaluation: no fruit test set is stored with the project, so the Test results and Curves tabs state that no evaluation is available.

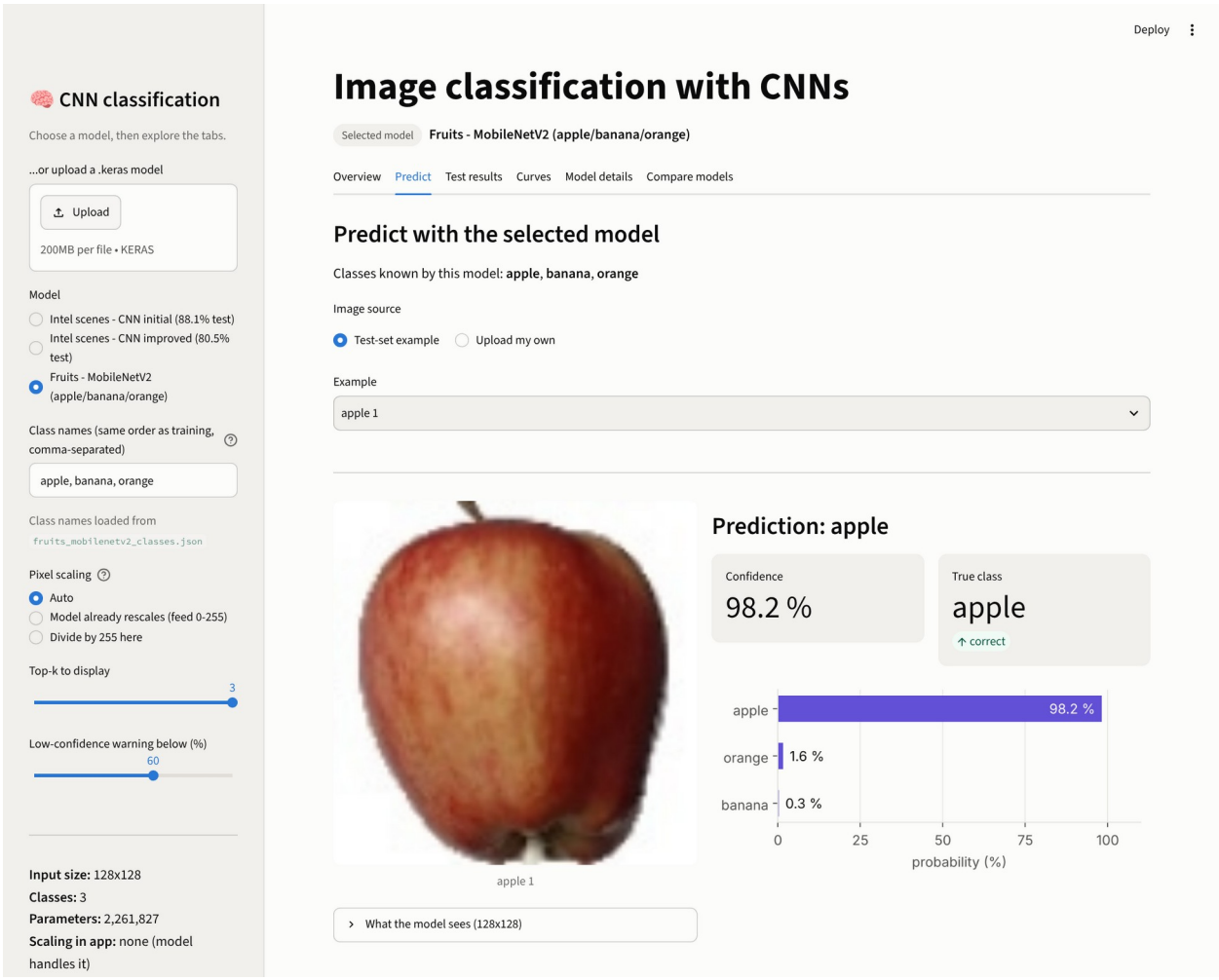


Figure 11 - Predict tab, fruit model: apple recognized with 98.2 % confidence.

4. Analysis: initial versus improved CNN

Measure	Initial CNN	Improved CNN
Parameters	101,894	406,694
Epochs / time	40 / 7.2 min	19 / 9.8 min
Validation accuracy	88.06 %	81.75 %
Test accuracy	88.10 %	80.53 %
Macro F1 (test)	88.24 %	80.63 %
Macro AUC (test)	0.985	0.972
Test errors (out of 3,000)	357	584
Images predicted with >= 90 % confidence	68.1 % (96.4 % correct)	55.7 % (93.8 % correct)
Main confusion	mountain -> glacier (76)	mountain -> sea (109)

- **More regularization did not help.** Batch normalization, dropout and augmentation made the model larger and slower to train but not better: it is 7.6 points less accurate on the test set, and the validation accuracy (81.75 %) already announced this gap.
- **The errors change nature.** The initial CNN mostly confuses visually close classes (mountain/glacier, buildings/street). The improved CNN also pushes many mountain and glacier images towards sea, a sign that the heavily regularized model under-fits these classes.
- **Confidence is informative.** With both models, high confidence goes with high accuracy, so the low-confidence warning of the interface is a useful signal; the initial CNN is simply more often confident and right.
- **Interface value.** Seeing the same image, the confusion matrix and the curves side by side makes these differences visible in a few clicks.

5. Limitations and notes

- **Training history.** The .keras files do not contain the per-epoch loss and accuracy. The Curves tab therefore shows ROC curves, calibration and confidence distribution (computed from the models), and offers a loader for the training history: exporting history.history from the notebook to results/<model>_history.json, or uploading it in the tab, displays the learning curves without any other change.
- **Model 3 missing.** The MobileNetV2 model for the Intel dataset is not in the folder; its figures come from the report.
- **Evaluation data.** Models 1 and 2 were re-evaluated with the same image loader as in training on the 3,000-image Intel test split; the accuracy and macro F1 match the report exactly. Example images in the Predict tab are taken from this test split (and, for the fruit model, from a public fruit image set), only for demonstration.
- **Closed set of classes.** As in the previous report, a model always answers with one of its own classes; the interface lists the known classes and warns when the confidence is low.

- **Single training run.** Each model was trained once, and with 3,000 test images the accuracy is known to about 1.3 points (95 % interval).

6. Conclusion

The new interface gathers, in one place, what the project produced: prediction with the initial and the improved CNN, their test results, their curves and their architecture, plus a direct comparison. It confirms with figures that the simple initial CNN (88.10 %) outperforms the more regularized CNN (80.53 %), and it keeps the dataset-independent design of the application, as the fruit model shows.

Appendix - Running the interface

```
pip install -r requirements.txt
streamlit run streamlit_cnn.py
```

Folder content: streamlit_cnn.py, the .keras models with their _classes.json files, results/ (test evaluation, predictions and report figures), samples/ (example images) and .streamlit/config.toml (theme).